

Ejercicios 2

Víctor Manuel Peiró Martínez

FIIS 3A

Ampliación de Bases de Datos

Problemas de agregación para la colección de users:

1. Edad media de los usuarios: Calcula la edad media de todos los usuarios.

```
exercises> db.users.aggregate([{$group : {_id : null, avg_age: {$avg: "$age"}}}])  
[ { _id: null, avg_age: 29.285714285714285 } ]
```

2. Los 5 usuarios más antiguos: Encuentre los 5 usuarios más antiguos.

```
exercises> db.users.aggregate([{$sort : {age: -1}},{$limit: 5}])  
[  
  {  
    _id: 10,  
    firstName: 'Hank',  
    lastName: 'Anderson',  
    email: 'hank.anderson@example.com',  
    age: 36,  
    username: 'hanka',  
    lastLogin: '2024-03-25'  
  },  
  {  
    _id: 2,  
    firstName: 'Jane',  
    lastName: 'Smith',  
    email: 'jane.smith@example.com',  
    age: 34,  
    username: 'janesmith',  
    lastLogin: '2024-08-15'  
  },  
  {  
    _id: 5,  
    firstName: 'Charlie',  
    lastName: 'Davis',  
    email: 'charliedavis@example.com',  
    age: 31,  
    username: 'charlied',  
    lastLogin: '2024-07-22'  
  },  
  {  
    _id: 4,  
    firstName: 'Bob',  
    lastName: 'Brown',  
    email: 'bob.brown@example.com',  
    age: 29,  
    username: 'bobb',  
    lastLogin: '2024-09-10'  
  },  
  {  
    _id: 1,  
    firstName: 'John',  
    lastName: 'Doe',  
    email: 'john.doe@example.com',  
    age: 28,  
    username: 'johndoe',  
    lastLogin: '2024-09-01'  
  }  
]
```

3. Usuarios con correos electrónicos de dominio específico: cuente el número de usuarios con direcciones de correo electrónico de un dominio específico (por ejemplo, example.com).

```
exercises> db.users.aggregate([{$match: {email:{$regex: /example.com$/}}},{$group: {_id: null, num_users: {$sum: 1}}}])  
[ { _id: null, num_users: 7 } ]
```

4. Inicios de sesión recientes: recupere los usuarios que han iniciado sesión en los últimos 30 días.

```
exercices> db.users.aggregate([{$addFields:{daysLastLogin:{$dateDiff:{startDate:{$toDate: '$lastLogin'}, endDate: '$NOW', unit: 'day'}}}},{$match: {daysLastLogin:{$lte:30}}}]
[
  {
    _id: 11,
    firstName: 'Victor',
    lastName: 'Peiro',
    email: 'victor.peiro@example.com',
    age: 20,
    username: 'victorp',
    lastLogin: '2025-10-01',
    daysLastLogin: Long('19')
  }
]
```

5. Nombres de usuario que comienzan con una letra: Encuentre todos los nombres de usuario que comiencen con una letra específica.

```
exercices> db.users.aggregate([{$match:{username:{$regex:/^j/}}}]
[
  {
    _id: 1,
    firstName: 'John',
    lastName: 'Doe',
    email: 'john.doe@example.com',
    age: 28,
    username: 'johndoe',
    lastLogin: '2024-09-01'
  },
  {
    _id: 2,
    firstName: 'Jane',
    lastName: 'Smith',
    email: 'jane.smith@example.com',
    age: 34,
    username: 'janesmith',
    lastLogin: '2024-08-15'
  }
]
```

6. Usuarios por último año de inicio de sesión: agrupe a los usuarios por el año de su último inicio de sesión y cuente el número de usuarios de cada año.

```
exercices> db.users.aggregate([
... {$group: {
... _id: {year:{$year:{$toDate : '$lastLogin'}}},
... num_users: {$sum:1}
... }}])
[
  { _id: { year: 2024 }, num_users: 6 },
  { _id: { year: 2025 }, num_users: 1 },
  { _id: { year: null }, num_users: 2 }
]
```

7. Usuarios con campos faltantes: identifique a los usuarios que no tienen una dirección de correo electrónico.

```
exercises> db.users.aggregate([ { $match: { email: null } } ] )
[
  { _id: ObjectId('68d11a5fd63e940bafce5f47') },
  {
    _id: ObjectId('68d67b3d982b6f61fb247938'),
    name: 'Janet',
    lastName: 'Smith'
  }
]
```

8. Edad media por nombre inicial: Calcula la edad media de los usuarios agrupados por la primera letra de su nombre.

```
exercises> db.users.aggregate([
... {$group:{
...   _id: {letra: {$substr:["$firstName",0,1]}},
...   edad_media: {$avg:"$age"}
... }}])
[
  { _id: { letra: '' }, edad_media: null },
  { _id: { letra: 'V' }, edad_media: 20 },
  { _id: { letra: 'J' }, edad_media: 31 },
  { _id: { letra: 'C' }, edad_media: 31 },
  { _id: { letra: 'E' }, edad_media: 27 },
  { _id: { letra: 'B' }, edad_media: 29 },
  { _id: { letra: 'H' }, edad_media: 36 }
]
```

9. Usuarios con varias condiciones: busque usuarios que tengan más de 30 años y que hayan iniciado sesión en los últimos 6 meses.

```
exercises> db.users.aggregate([
...   {$addFields: { monthsLogin: {$dateDiff: {startDate: { $toDate:"$lastLogin"},endDate:"$$NOW",unit:"month"}}}},
...   {$match: {monthsLogin:{$lt:6},age:{$gte:30}}}
... ])
[
  {
    _id: ObjectId('68d67b3d982b6f61fb247938'),
    name: 'Janet',
    lastName: 'Smith',
    age: 35,
    lastLogin: '2025-07-22',
    monthsLogin: Long('3')
  }
]
```

Problemas de agregación para la colección de products

11. Precio medio por categoría: Calcula el precio medio de los productos de cada categoría.

```
exercises> db.products.aggregate([
... {$group: {
...   _id: "$category",
...   avg_price: {$avg: "$price"}
... }}])
[
  { _id: 'Electronics', avg_price: 806.6666666666666 },
  { _id: 'Accessories', avg_price: 385.00000000000006 },
  { _id: 'Sportswear', avg_price: 126.50000000000001 },
  { _id: 'Home Appliances', avg_price: 137.5 },
  { _id: 'Furniture', avg_price: 302.5 }
]
exercises> |
```

12. Valor total del inventario: Calcule el valor total de todos los productos.

```
exercises> db.products.aggregate([
... {$group: {
...   _id : null,
...   total_price : {$sum: "$price"}
... }}])
[ { _id: null, total_price: 4064.5000000000005 } ]
exercises> |
```

13. Los 3 productos más caros: Encuentra los 3 productos más caros.

```
exercises> db.products.aggregate([
... {$sort: {price:-1}},{$limit:3}
... ])
[
  {
    _id: 1,
    name: 'Laptop',
    description: 'A high-performance laptop',
    price: 1320,
    category: 'Electronics'
  },
  {
    _id: 2,
    name: 'Smartphone',
    description: 'A latest model smartphone',
    price: 880.0000000000001,
    category: 'Electronics'
  },
  {
    _id: 10,
    name: 'Watch',
    description: 'Smartwatch with fitness tracking',
    price: 385.00000000000006,
    category: 'Accessories'
  }
]
exercises> |
```

14. Productos con una palabra clave específica: busque productos cuya descripción contenga una palabra clave específica.

```
exercises> db.products.aggregate([
... {$match: {description: {$regex: /phone/}}}
... ])
[
  {
    _id: 2,
    name: 'Smartphone',
    description: 'A latest model smartphone',
    price: 880.00000000000001,
    category: 'Electronics'
  },
  {
    _id: 3,
    name: 'Headphones',
    description: 'Noise-cancelling headphones',
    price: 220.00000000000003,
    category: 'Electronics'
  }
]
```

15. Recuento de productos por categoría: cuente el número de productos de cada categoría.

```
exercises> db.products.aggregate([
... {$group: {
... _id: "$category",
... num_productos: {$sum: 1}
... }}}]
[
  { _id: 'Home Appliances', num_productos: 2 },
  { _id: 'Sportswear', num_productos: 3 },
  { _id: 'Accessories', num_productos: 1 },
  { _id: 'Electronics', num_productos: 3 },
  { _id: 'Furniture', num_productos: 2 }
]
```

16. Simulación de aumento de precios: Calcule los nuevos precios si los precios de todos los productos se incrementaron en un 10%.

```
exercises> db.products.aggregate([
... {$project: {
... name : 1,
... new_price: {$multiply:["$price",1.1]}
... }}}]
[
  { _id: 1, name: 'Laptop', new_price: 1452.0000000000002 },
  { _id: 2, name: 'Smartphone', new_price: 968.0000000000002 },
  { _id: 10, name: 'Watch', new_price: 423.5000000000001 },
  { _id: 5, name: 'Blender', new_price: 181.50000000000003 },
  { _id: 3, name: 'Headphones', new_price: 242.00000000000006 },
  { _id: 4, name: 'Coffee Maker', new_price: 121.00000000000003 },
  { _id: 8, name: 'Running Shoes', new_price: 145.20000000000002 },
  { _id: 6, name: 'Desk Chair', new_price: 302.5 },
  { _id: 9, name: 'Jacket', new_price: 217.80000000000004 },
  { _id: 7, name: 'Bookshelf', new_price: 363.00000000000006 },
  { _id: 11, name: 'T-Shirt', new_price: 54.45000000000001 }
]
```

17. Productos con varias condiciones: busca productos que pertenezcan a una categoría específica y que tengan un precio superior a un determinado umbral.

```
exercises> db.products.aggregate([
... {$match:
... {category:"Sportswear",price:{$gte:100}}
... }])
[
  {
    _id: 8,
    name: 'Running Shoes',
    description: 'Comfortable running shoes',
    price: 132,
    category: 'Sportswear'
  },
  {
    _id: 9,
    name: 'Jacket',
    description: 'Waterproof jacket',
    price: 198.00000000000003,
    category: 'Sportswear'
  }
]
```

Problemas de agregación para la colección de places

21. Calificación promedio de los lugares: Calcule la calificación promedio de todos los lugares.

```
exercises> db.places.aggregate([
...   {$group: {
...     _id: "rating",
...     avg_rating: {$avg: "$rating"}
...   }}])
[ { _id: 'rating', avg_rating: 4.76 } ]
```

22. Lugares cerca de un punto: Encuentre todos los lugares dentro de un radio de 10 kilómetros de un punto determinado.

```
exercises> db.places.aggregate([ {$geoNear: { near: { type: "Point", coordinates: [-73.8650, 40.7820] }, maxDistance: 10000, distanceField: "distance", spherical: true } }])
[ {
  _id: 1,
  name: 'Central Park',
  location: { type: 'Point', coordinates: [ -73.9654, 40.7829 ] },
  rating: 4.7,
  distance: 8463.317728147455
}
```

23. Los 5 lugares mejor valorados: Encuentra los 5 lugares mejor valorados.

```
exercises> db.places.aggregate([
...   {$sort : {rating: -1}}, {$limit: 5}
... ])
[
  {
    _id: 3,
    name: 'Louvre Museum',
    location: { type: 'Point', coordinates: [ 2.3364, 48.8606 ] },
    rating: 4.9
  },
  {
    _id: 8,
    name: 'Taj Mahal',
    location: { type: 'Point', coordinates: [ 78.0421, 27.1751 ] },
    rating: 4.9
  },
  {
    _id: 2,
    name: 'Eiffel Tower',
    location: { type: 'Point', coordinates: [ 2.2945, 48.8584 ] },
    rating: 4.8
  },
  {
    _id: 9,
    name: 'Machu Picchu',
    location: { type: 'Point', coordinates: [ -72.545, -13.1631 ] },
    rating: 4.8
  },
  {
    _id: 6,
    name: 'Great Wall of China',
    location: { type: 'Point', coordinates: [ 116.5704, 40.4319 ] },
    rating: 4.8
  }
]
```

24. Lugares por proximidad: ordene los lugares por su distancia a un punto determinado.

```
exercises> db.places.aggregate([
...   {$geoNear: {
...     near: {type: "Point", coordinates: [0,0]},
...     distanceField: "distanceCalc",
...     spherical: true}
...   }]
... )
[
  {
    _id: 5,
    name: 'Colosseum',
    location: { type: 'Point', coordinates: [ 12.4922, 41.8902 ] },
    rating: 4.7,
    distanceCalc: 4829128.852056703
  },
  {
    _id: 10,
    name: 'Christ the Redeemer',
    location: { type: 'Point', coordinates: [ -43.2105, -22.9519 ] },
    rating: 4.7,
    distanceCalc: 5326016.119249308
  },
  {
    _id: 2,
    name: 'Eiffel Tower',
    location: { type: 'Point', coordinates: [ 2.2945, 48.8584 ] },
    rating: 4.8,
    distanceCalc: 5443326.78788255
  },
  {
    _id: 3,
    name: 'Louvre Museum',
    location: { type: 'Point', coordinates: [ 2.3364, 48.8606 ] },
    rating: 4.9,
    distanceCalc: 5443735.857620721
  },
  {
    _id: 9,
    name: 'Machu Picchu',
    location: { type: 'Point', coordinates: [ -72.545, -13.1631 ] },
    rating: 4.8,
    distanceCalc: 8128250.9547565775
  },
]
```

25. Lugares con una palabra clave de nombre específico: busque lugares cuyo nombre contenga una palabra clave específica.

```
exercises> db.places.aggregate([
...   {$match : {name: {$regex: /of/}}}
... ])
[
  {
    _id: 4,
    name: 'Statue of Liberty',
    location: { type: 'Point', coordinates: [ -74.0445, 40.6892 ] },
    rating: 4.6
  },
  {
    _id: 6,
    name: 'Great Wall of China',
    location: { type: 'Point', coordinates: [ 116.5704, 40.4319 ] },
    rating: 4.8
  }
]
```

26. Lugares por recuento de calificación: cuente el número de lugares para cada valor de calificación.

```
exercises> db.places.aggregate([
...   {$group: {
...     _id: "$rating",
...     num_places: {$sum: 1}
...   }}])
[
  { _id: 4.6, num_places: 1 },
  { _id: 4.9, num_places: 2 },
  { _id: 4.7, num_places: 4 },
  { _id: 4.8, num_places: 3 }
]
exercises> |
```

27. Lugares con múltiples condiciones: Encuentre lugares que tengan una calificación superior a 4.5 y estén a una cierta distancia de un punto determinado

```
exercises> db.places.aggregate([
...   {$geoNear: {
...     near: {type: "Point", coordinates: [2.2945, 48.8584]},
...     maxDistance: 10000,
...     distanceField: "distanceCalculated",
...     spherical: true}},
...   {$match: {rating: {$gte : 4.5}}}
... ])
[
  {
    _id: 2,
    name: 'Eiffel Tower',
    location: { type: 'Point', coordinates: [ 2.2945, 48.8584 ] },
    rating: 4.8,
    distanceCalculated: 0
  },
  {
    _id: 3,
    name: 'Louvre Museum',
    location: { type: 'Point', coordinates: [ 2.3364, 48.8606 ] },
    rating: 4.9,
    distanceCalculated: 3078.409450014034
  }
]
```